NEUROQUANT / QUANTEDGE

Monorepo Architecture & Full Project KT Guide

Architecture reconstructed from the uploaded repository snapshot - 27 Aug 2026

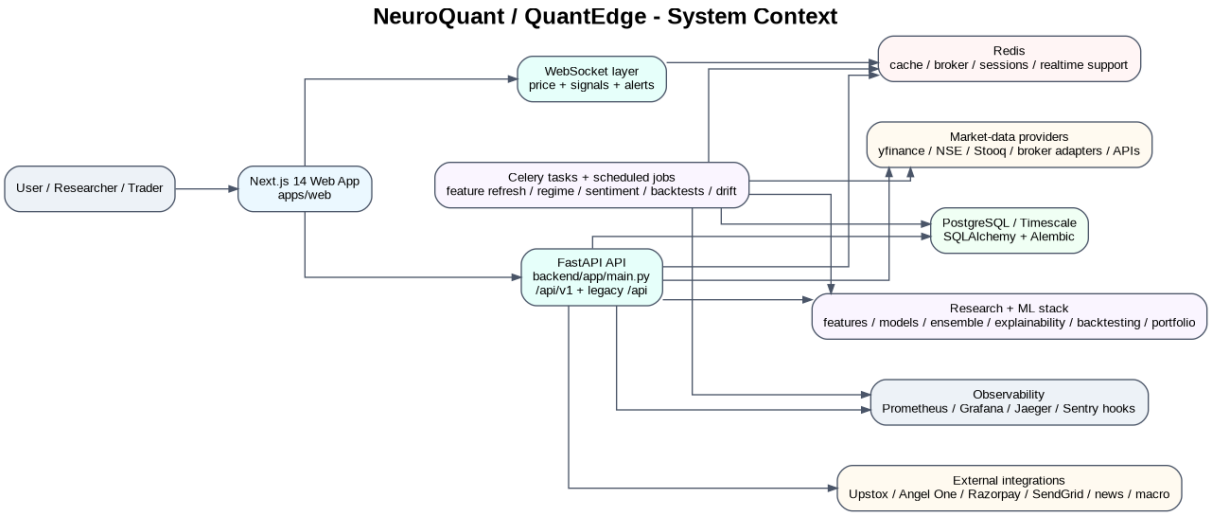


Figure 1. System context and major runtime boundaries.

Purpose: give a new engineer enough architectural context to navigate the project, trace a feature end-to-end, and know where to make changes safely.

1. KT at a Glance

This document is a handoff-oriented architecture guide, not a marketing diagram. It reflects the files present in the uploaded ZIP, including the current working-tree state. It distinguishes the implemented paths from optional or auxiliary infrastructure.

Area	Primary location	What it owns
Web product	apps/web	Next.js UI, route pages, components, hooks, API clients, realtime UI state
Shared frontend packages	packages/ui, packages/types, packages/config	Reusable React UI, TS contracts/declarations, shared config
API / business backend	backend/app	FastAPI routes, auth, market data, signals, predictions, trading, portfolio, payments
Research / ML	backend/research	Features, models, ensemble, explainability, backtesting, portfolio optimization
Background processing	backend/app/tasks, backend/app/core/celery_app.py	Scheduled/async work: data, features, regime, sentiment, backtests, drift/retraining
Persistence	backend/app/database, backend/app/models, alembic	PostgreSQL/SQLite dev DB, SQLAlchemy models, migrations
External integrations	backend/app/providers, services	Market/broker/payment/news/email integrations
Operations	docker-compose.yml, infrastructure/	Nginx, DB, Redis, observability and supporting containers
Auxiliary service	services/gateway	Separate Python gateway surface; not the main FastAPI runtime path

2. The Mental Model a New Engineer Should Use

Think of the repository as one product with four major planes:

- Experience plane: Next.js pages/components/hooks render dashboards, terminal, research, portfolio, billing and admin surfaces.
- Application plane: FastAPI exposes authenticated REST endpoints and WebSockets; service modules implement market, signal, prediction, portfolio and trading behavior.
- Research plane: backend/research contains reusable ML/statistical modules. It feeds inference/backtesting workflows but is not itself a standalone web service.
- Operations/data plane: PostgreSQL/Redis plus provider integrations and Celery jobs keep the application stateful, scheduled and observable.

The important architectural rule for KT is: do not treat every Python file as an independent microservice. Most backend Python code runs in a single FastAPI process, while Celery provides background execution and the external containers provide infrastructure.

3. Monorepo Structure

```
stock-market/
├── apps/web/                # Next.js 14 application
├── packages/ui/             # shared UI components
├── packages/types/          # shared TS types/API declarations
├── packages/config/         # shared TS/Tailwind/Next config
```

```
├── services/gateway/         # auxiliary Python gateway
├── backend/
│   ├── app/                  # FastAPI runtime + services
│   ├── research/             # ML/research library
│   ├── tasks/                # alternate/legacy task layer
│   ├── scripts/              # operational / research scripts
│   ├── alembic/              # DB migrations
│   └── tests/                # backend test suite
├── infrastructure/           # nginx, postgres, redis, prometheus config
├── data/                     # local data artifacts
├── docs/                     # supplemental docs
├── archive/                  # archived material
├── artifacts/                # generated/local artifacts
├── docker-compose.yml
├── turbo.json
├── pnpm-workspace.yaml
└── package.json
```

Path	Navigation meaning	Key files/folders
apps/web/src/app	User-facing route map	(app), (auth), markets, terminal, research, portfolio, backtest-lab
apps/web/src/components	UI assembly layer	terminal, dashboard, charts, layout, market, alerts
apps/web/src/hooks	Client-side orchestration	useTerminalData, usePriceFeed, useSignalFeed, query hooks
apps/web/src/lib	API/runtime integrations	api clients, contracts, runtime-config, auth helpers
backend/app/api/v1	REST contract surface	auth, market, signals, predictions, regime, portfolio, paper_trading, trading, payments, etc.
backend/app/services	Domain/application services	market_data, signal_engine, prediction_engine, regime_analyzer, paper_trading, risk_engine, etc.
backend/app/models	Persistence model layer	user, instrument, OHLCV, signal, prediction, portfolio, payment, model metadata
backend/research	Research computation	feature_engineering, models, backtesting, explainability, portfolio, risk
backend/app/tasks	Background jobs used by current app	market data, features, regime, sentiment, backtests, retraining trigger

4. Runtime / Container Architecture

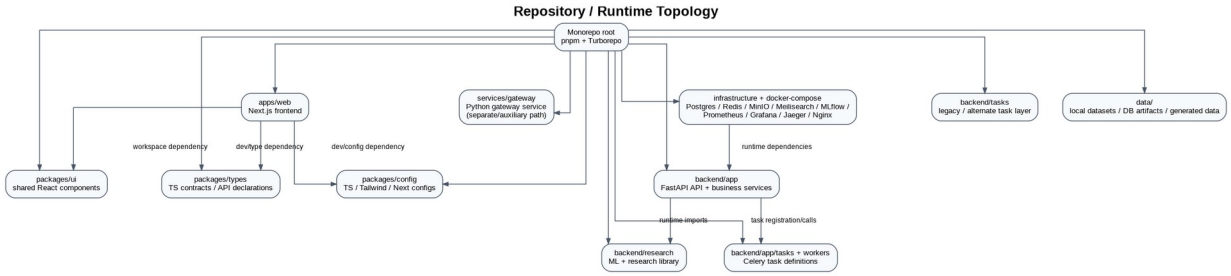


Figure 2. Monorepo package and runtime boundaries.

The repository is a monorepo managed with pnpm + Turbo. The web app consumes workspace packages. The backend is independently built with Python dependencies and is containerized from backend/Dockerfile. docker-compose.yml additionally declares the stateful/observability services.

Runtime	Port in compose	Role	Notes
frontend	3000	Next.js web application	Built from root with Dockerfile.web; app depends on backend
backend	8000	FastAPI application	Uvicorn starts app.main:app
postgres	5432	Primary relational persistence	Timescale image; SQLAlchemy/asyncpg
redis	6379	Cache/broker/session/ realtime support	Celery broker/backend settings use Redis URLs
nginx	80/443	Reverse proxy / edge	Configuration under infrastructure/nginx
minio	9000/9001	Object storage support	Declared in compose; artifact storage path is not the only model-artifact mechanism
meilisearch	7700	Search service	Declared infrastructure dependency
mlflow	5000	Experiment/model tracking support	Declared as supporting infrastructure
prometheus	9090	Metrics collection	infrastructure/prometheus/ prometheus.yml
grafana	3001 -> 3000	Metrics dashboards	paired with Prometheus
jaeger	16686	Tracing UI	OpenTelemetry/trace support

5. Backend Entry Point and Request Pipeline

The main runtime starts in backend/app/main.py. That file creates the FastAPI application, configures middleware, imports the Celery app early for task registration, mounts the v1 API router, the QuantEdge academic router, and WebSocket routes, and runs startup/shutdown hooks.

```
Browser
  -> Next.js route / client hook
  -> same-origin /api/v1 proxy OR configured API base
  -> backend/app/main.py
  -> middleware (request id, security headers, rate limit, CORS/trusted host)
  -> /api/v1 router
  -> endpoint module
  -> domain/application service
  -> data provider / DB / Redis / research module
  -> response schema
  -> browser state / page
```

5.1 API surfaces

Surface	Mount	Purpose
Modern REST	/api/v1	Primary application API
Academic / compatibility REST	/api	Explicit QuantEdge endpoints such as /api/ohlc/{symbol}, /api/indicators/{symbol}, /api/signals/{symbol}, /api/regime/{symbol}, /api/backtest, /api/paper-trade
WebSocket	WS routes in backend/app/websocket	Live price, signal and alert delivery
Health / readiness	/api/v1/health...	Operational checks
OpenAPI	FastAPI-generated + checked-in openapi.json	Contract discovery and frontend development reference

5.2 REST route map

Domain	Router file	Representative endpoints
Auth	api/v1/auth.py	register, login, refresh, logout, me, forgot/reset password
Market	api/v1/market.py	quote, history, indices, movers, heatmap, search, universe, provider health
Signals	api/v1/signals.py	bulk signals, single signal, signal history
Predictions	api/v1/predictions.py	prediction, forecast, latest, ensemble, history
Regime	api/v1/regime.py	current, history, statistics
Intelligence	api/v1/intelligence.py	economic calendar, correlation graph, factor exposure, monitoring/explainability aliases
Backtest	api/v1/backtest.py	run, status, results
Portfolio	api/v1/portfolio.py	portfolios, holdings, performance, transactions, risk metrics, optimization
Paper trading	api/v1/paper_trading.py	order, positions, history, P&L, cancel, reset
Live trading	api/v1/trading.py	mode, order, kill switch, audit log, Upstox OAuth/status
Alerts	api/v1/alerts.py	list/create/update/delete/history
Payments	api/v1/payments.py	plans, balance, intents, confirm, subscribe, history, subscription, webhook
Users / watchlists	users.py / watchlist.py	profile and symbol list management
Research explanation	api/v1/explain.py	SHAP, attention, counterfactual
Model monitoring	api/v1/models.py	accuracy, drift, ensemble weight history
Admin / blog / notifications / journal	respective routers	operational and product features

6. Frontend Architecture

The web application is a Next.js 14 App Router application. Authenticated routes sit under apps/web/src/app/(app), while login/registration flows are under (auth). The authenticated root redirects to /dashboard in the current snapshot; /terminal is the main trading-focused surface.

Frontend area	Examples	Responsibility
App routes	dashboard, terminal, markets, research, portfolio, backtest-lab, model-monitor, signals	Page-level composition
Components	components/terminal, charts, dashboard, market, layout	Reusable visual/interaction building blocks
Modules	modules/auth, dashboard, signals, payments, admin	Feature-oriented logic
Hooks	useTerminalData, usePriceFeed, useSignalFeed, query hooks	Client orchestration and realtime state
API clients	src/lib/*-api.ts, contracts-api.ts	REST calls and response adaptation
Shared UI	packages/ui	Cross-page visual primitives
Shared contracts	packages/types	TypeScript declarations and API contract helpers
State/data libraries	TanStack Query + Zustand	Server cache and client state

7. Live Terminal Architecture

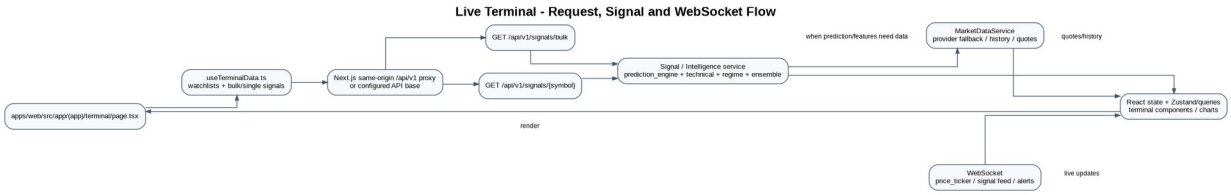


Figure 3. Terminal data lifecycle as implemented in the current frontend/backend split.

The terminal is a good example of the project's hybrid data pattern: initial state arrives over REST, then the UI is incrementally refreshed through polling and WebSocket updates. apps/web/src/hooks/useTerminalData.ts loads four watchlist buckets, de-duplicates symbols, calls bulk and single signal endpoints, and then reconciles incoming live signals into the displayed rows.

Step	Code location	Behavior
1	terminal/page.tsx + TerminalLayout	Mount authenticated terminal surface
2	useTerminalData.ts	Load watchlists; fallback to default symbol set if unavailable
3	intelligence-api.ts / related clients	Fetch bulk and selected-symbol signal data
4	backend/api/v1/signals.py + intelligence.py	Resolve signals through backend service logic
5	prediction_engine / signal_engine / regime	Compute or assemble technical + model + regime signal data
6	useSignalFeed / usePriceFeed	Maintain live singleton WebSocket streams
7	Terminal components	Merge REST baseline with live websocket deltas and render charts/panes

8. Market Data Layer

The backend deliberately has multiple data-access paths. The central production-facing abstraction is MarketDataService and the provider/source classes under app/services/data_ingestion and app/providers. The repository also contains direct service integrations such as yfinance, NSE, Stooq, Angel One and Upstox adapters.

Component	Location	Role
MarketDataService	backend/app/services/ market_data_service.py	Quotes/history/provider health and higher-level market data access
Provider manager / sources	backend/app/services/data_ingestion/	Provider abstraction and fallback behavior
NSE source	data_ingestion/nse_source.py	NSE-oriented source path
Stooq / yfinance	data_ingestion/*.py	Fallback/historical market data paths
Angel One	app/providers/angel_one_provider.py	Broker/market-data integration
Upstox	app/services/upstox_service.py + trading router	OAuth/trading integration
Instrument universe	models/instrument.py + universe scripts	Canonical symbol/instrument records

Important KT caveat: the repository documentation itself notes that the current setup should not be represented as an exchange-grade, guaranteed 10,000+ real-time feed. A future scale-out needs a canonical instrument master, a dedicated market-data provider strategy, streaming fan-out, backpressure, health monitoring and replay/recovery.

9. Research and ML Architecture

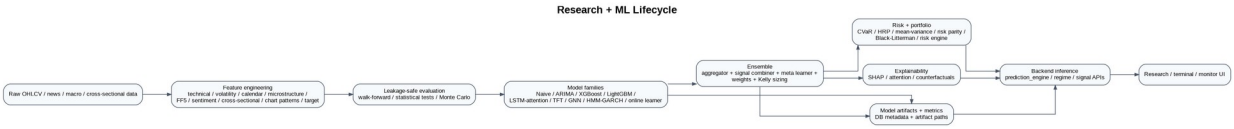


Figure 4. Research, model training, evaluation, inference and explainability lifecycle.

Layer	Directory	Representative modules
Feature engineering	backend/research/feature_engineering	technical, volatility, calendar, microstructure, FF5, sentiment, cross-sectional, chart patterns, target, pipeline
Baselines	models/baselines	naive, ARIMA
Tree models	models/xgboost_model, lightgbm_model	classifiers
Deep sequence models	models/lstm_attention, models/tft	LSTM-attention, Temporal Fusion Transformer
Graph models	models/gnn	graph builder, dataset, trainer, inference, contagion risk
Regime / volatility	models/hmm_garch	HMM, GARCH, forecaster, regime detector
Online learning	models/online_learner	adaptive learner
Ensemble	models/ensemble	orchestrator, aggregator, signal combiner, meta learner, weights, Kelly sizer
Backtesting	research/backtesting	engine, cost model, walk-forward, analytics, Monte Carlo, statistical tests
Explainability	research/explainability	SHAP, attention extraction, counterfactual
Portfolio / risk	research/portfolio, research/portfolio_optimization, research/risk	HRP, CVaR, MVO, risk parity, Black-Litterman and risk calculations

Inference is exposed through the backend service layer rather than through a separate model-serving microservice. For example, predictions.py, intelligence.py and the QuantEdge academic router call service functions such as prediction_engine / detect_regime / explainability helpers. Model metadata and metrics are represented in SQLAlchemy models ModelArtifact and ModelMetric.

10. Model Lifecycle and MLOps

1. Train or evaluate models using backend/scripts and backend/research/model modules.

2. Persist model metadata in model_artifact (model type, version tag, artifact path, hyperparameters, active flag).

3. Persist evaluation statistics in model_metric (accuracy, precision, recall, AUC, Sharpe, cost-adjusted Sharpe, regime breakdown).

4. Use ensemble components to combine model outputs and apply weights / Kelly sizing.

5. Serve predictions through backend inference endpoints and use model-monitoring endpoints for accuracy, drift and ensemble-weight history.

6. Use scheduled drift checks to decide whether retraining should be triggered.

Current Celery beat definitions in backend/app/core/celery_app.py include hourly news sentiment refresh, daily regime-state updates and daily drift checks. In development, task_always_eager is enabled when ENVIRONMENT is development; this changes the operational behavior versus a dedicated worker deployment.

11. Background Jobs and Scheduling

Task	Location	Cadence / trigger	Purpose
update_news_sentiment	app/tasks/sentiment_tasks.py	hourly beat	ingest/update news sentiment
update_regime_states	app/tasks/regime_tasks.py	daily beat	refresh market regime states
check_drift_for_watchlist	app/tasks/retraining_trigger.py	daily beat	drift detection + retraining decision
build_feature_vectors	app/tasks/feature_tasks.py	on demand/scheduled workflows	feature refresh
market_data.*	app/tasks/market_data_tasks.py	operational scripts / jobs	seed universe and historical data
run_backtest_job_task	app/tasks/backtest_tasks.py	API-triggered background job	asynchronous backtest execution

There is also backend/tasks as a second/legacy task layer. A new engineer should verify imports and the active task registration before adding jobs there; current app startup explicitly imports backend/app/tasks/retraining_trigger so the intended Celery task registration is not silently omitted.

12. Data Model - Core Business Entities

Entity	Model	Relationship / why it matters
User	models/user.py	authentication, profile and ownership context
Instrument	models/instrument.py	symbol/instrument master reference
OHLCV	models/ohlcv.py	historical market bars
FeatureVector	models/feature_vectors.py	stored feature representations
Prediction	models/prediction.py	model inference outputs
Signal	models/signal.py	signal history / actionable output
RegimeState	models/regime_states.py	regime classification and state history
ModelArtifact	models/model_artifact.py	trained model metadata and artifact path
ModelMetric	models/model_metric.py	out-of-sample model evaluation
Portfolio / transactions	models/portfolio.py, transactions.py	holdings, cash movement and transactions
PaperTrading	models/paper_trading.py	paper orders/positions/P&L state
Alerts	models/alert.py	user alert definitions/history
Payments	models/payment.py	subscription/payment records

Journal	models/journal.py	user trade/research notes

13. Security Architecture

Security is cross-cutting and implemented primarily under backend/app/core plus auth/router dependencies.

- JWT uses RSA/RS256 with private/public PEM keys; the current code expects paths such as ./keys/private.pem and ./keys/public.pem.
- Passwords are configured for Argon2/bcrypt-style hashing and minimum-complexity policy.
- CSRF, rate limiting, request IDs, security headers, trusted-host validation and CORS are configured in the FastAPI layer.
- Payments and broker integrations are authenticated separately and should be treated as privileged boundaries.
- Upstox order flow includes a server-side per-order value cap configuration.
- The repository snapshot contains local .env files and credential-bearing configuration; do not copy those values into documentation or commits. Rotate exposed credentials if this archive has been shared outside a trusted boundary.

14. Deployment / Operations

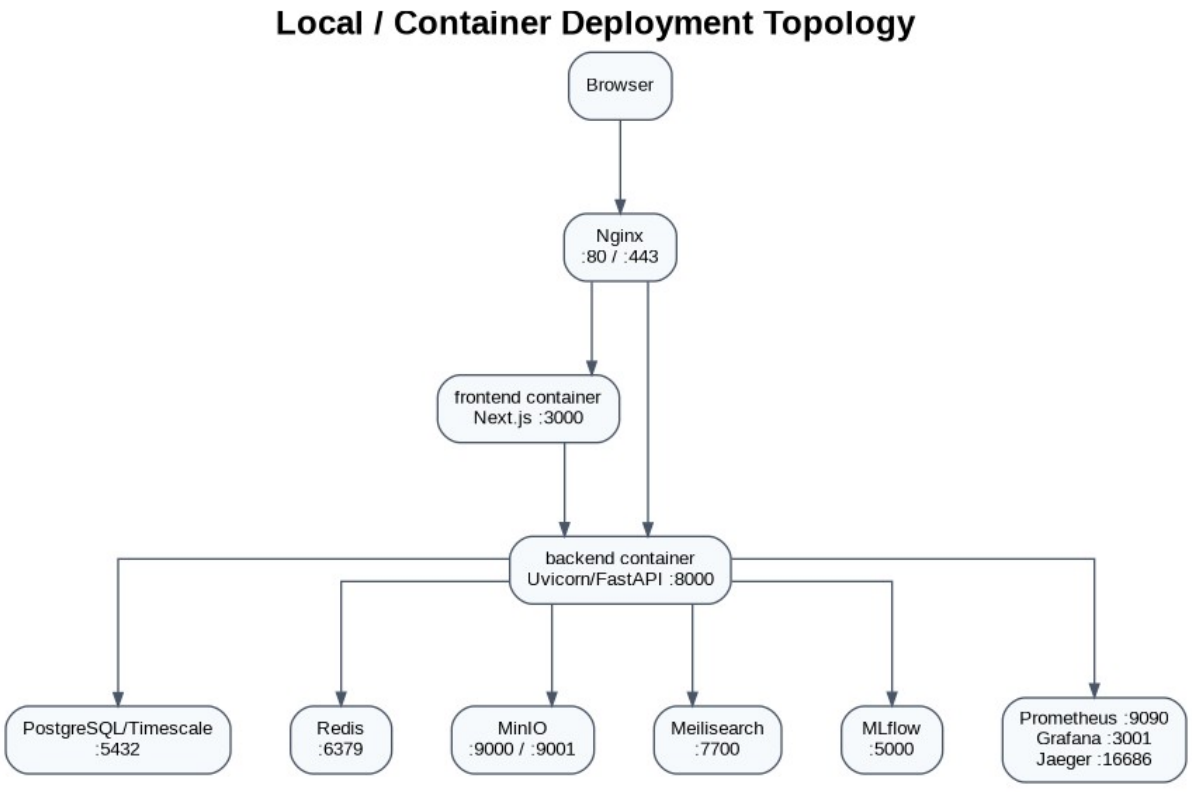


Figure 5. docker-compose deployment topology declared by the repository.

The canonical local/container story is defined by docker-compose.yml. The frontend is built from the repository root; the backend image is built from backend/. Nginx is present as an edge proxy. The compose file also declares

observability and supporting infrastructure, but not every declared container is necessarily required for every developer workflow.

15. Configuration Map

Configuration	Primary source	Examples / notes
Backend settings	backend/app/core/config.py + .env	DATABASE_URL, REDIS_URL, JWT paths, provider keys, CORS, feature flags
Frontend runtime API	apps/web/src/lib/runtime-config.ts + env	API base and WebSocket URL selection; local browser prefers same-origin proxy in the current flow
Frontend build	apps/web/package.json / next.config.js / tsconfig	Next.js 14, TypeScript, Tailwind, React Query, Zustand
Monorepo	package.json / pnpm-workspace.yaml / turbo.json	pnpm workspaces + Turbo tasks
DB migrations	backend/alembic	schema changes and migration environment
Infra configs	infrastructure/*	nginx, postgres init, redis, prometheus

16. Critical End-to-End Flows for KT

16.1 Registration / Login

```
/register -> backend /api/v1/auth/register -> user persistence -> token response / auth state
/login -> /api/v1/auth/login -> JWT/refresh token flow -> authenticated App Router shell -> /dashboard
```

16.2 Market Quote / History

```
UI -> /api/v1/market/quote/{symbol} or /history/{symbol}
   -> MarketDataService
   -> provider/source selection
   -> normalize result
   -> schema -> UI/chart
```

16.3 Signal

```
UI -> /api/v1/signals/{symbol}
   -> signal/intelligence endpoint
   -> prediction_engine / signal_engine
   -> market data + feature/regime/model outputs
   -> ensemble signal + confidence + risk levels
   -> SignalResponse
   -> terminal / signals UI
```

16.4 Backtest

```
UI -> POST /api/v1/backtest/run
   -> create job
   -> background task execute_backtest_job / run_backtest_job_task
   -> research.backtesting engine
   -> analytics / statistical tests / Monte Carlo
   -> persist status/results
   -> GET status/results -> UI
```

16.5 Paper / Live Trading

```
UI -> paper_trading endpoint OR trading endpoint
   -> auth/entitlement/risk checks
   -> PaperTradingService OR BrokerAdapter / Upstox integration
   -> transaction/order/audit state
   -> response + realtime updates
```

17. Where to Change What

Change needed	Start here	Then inspect
Add/change REST endpoint	backend/app/api/v1/<domain>.py	schema + service + model/test
Change signal logic	backend/app/services/ signal_engine.py / signals_service.py	prediction_engine, regime_analyzer, ensemble
Change prediction fusion	backend/app/services/ prediction_engine.py	backend/research/models/ensemble/*
Add ML feature	backend/research/feature_engineering/	target/pipeline + backtest leakage tests
Add/change model	backend/research/models/	training/evaluation/inference + model artifact/metric handling
Change explainability	backend/research/explainability/ + backend/app/api/v1/explain.py	UI research explanation pages
Change market provider	backend/app/services/data_ingestion/ or providers/	MarketDataService + health/fallback behavior
Change terminal behavior	apps/web/src/hooks/ useTerminalData.ts + components/terminal/	useSignalFeed/usePriceFeed + API clients
Change shared UI	packages/ui/	web imports and visual regression
Change persistence	backend/app/models + alembic/	repositories/services + migration
Change scheduled jobs	backend/app/tasks + backend/app/core/celery_app.py	worker deployment and task registration
Change reverse proxy / infra	infrastructure/ + docker-compose.yml	frontend/backend port/config expectations

18. Known Architectural Caveats New Engineers Should Know

- The repo contains both modern and compatibility/academic endpoint surfaces. Do not create a third API pattern without a deliberate reason.
- There is an auxiliary services/gateway package in addition to the main backend/app FastAPI runtime. Confirm whether a feature actually belongs there before editing it.
- There are two task namespaces: backend/app/tasks and backend/tasks. The current FastAPI startup explicitly imports the app/tasks path for task registration.
- The backend inference path can perform synchronous external data work. The repository notes that prediction and portfolio-performance calls can take several seconds in some local scenarios; concurrency/worker strategy matters.
- The project has a strong research layer, but production claims should follow measured provider/model behavior rather than directory names alone.
- Local development can run with SQLite/dev DB artifacts and eager Celery behavior even though the containerized target uses PostgreSQL/Redis. Always check environment-specific behavior when debugging a defect.
- The Git working tree in the supplied snapshot contains many modifications relative to main. A future engineer should preserve the exact snapshot or branch before large changes.

19. First-Day KT Checklist

1. Read README.md, ARCHITECTURE.md, KT_DOCUMENT.md and LIMITATIONS.md to understand project intent and known issues.

2. Start backend/app/main.py, backend/app/api/v1/router.py and backend/app/core/config.py.

3. Walk one domain end-to-end; terminal/signal is recommended because it touches frontend, API, market data, ML and WebSockets.

4. Read apps/web/src/lib/runtime-config.ts and the relevant API client before changing a frontend/backend call.

5. Understand PostgreSQL/Redis expectations and the difference between local eager Celery and real workers.

6. Inspect backend/research/models/ensemble plus prediction_engine before modifying signal fusion.

7. Run the existing tests/lint/build workflow before making changes so baseline failures are recorded.

8. Never commit secrets from .env files or local key material.

20. Recommended KT Walkthrough Order

1. Product surfaces: dashboard -> terminal -> market -> signals -> research -> portfolio -> backtest -> billing/admin.

2. Frontend internals: route -> component -> hook/query -> API client -> realtime hook.

3. Backend internals: router -> schema -> service -> model/provider/research module.

4. Async internals: endpoint -> task -> Celery configuration -> persistence/result retrieval.

5. ML internals: raw data -> features -> model -> ensemble -> risk -> explanation -> endpoint -> UI.

6. Operations: docker-compose -> Nginx -> FastAPI/Next.js -> PostgreSQL/Redis -> observability.

Appendix A. Source-of-Truth File Map

Topic	Source of truth
Application entrypoint	backend/app/main.py
REST router aggregation	backend/app/api/v1/router.py
Academic compatibility API	backend/app/api/quantedge_router.py
Backend settings	backend/app/core/config.py
Celery configuration	backend/app/core/celery_app.py
Database session/engine	backend/app/database/connection.py
Redis client	backend/app/database/redis_client.py
Frontend runtime config	apps/web/src/lib/runtime-config.ts
Terminal orchestration	apps/web/src/hooks/useTerminalData.ts
Signal engine	backend/app/services/signal_engine.py / signals_service.py
Prediction engine	backend/app/services/prediction_engine.py
Market data	backend/app/services/market_data_service.py
Research features	backend/research/feature_engineering/
Research models	backend/research/models/
Backtesting	backend/research/backtesting/
Explainability	backend/research/explainability/
Portfolio/risk research	backend/research/portfolio/ and backend/research/risk/
DB schema history	backend/alembic/
Container topology	docker-compose.yml
Reverse proxy	infrastructure/nginx/nginx.conf

Appendix B. Current Repository-State Notes

This KT is based on the uploaded 27 Aug 2026 ZIP. The snapshot is a working-tree state rather than a clean historical release: Git metadata indicates the current branch is main with substantial tracked-file modifications relative to the same main commit baseline. The diagrams therefore describe the code that is actually present in the archive, not a hypothetical clean-room architecture.

Security-sensitive files were intentionally not reproduced in this document. Local environment files, private keys, tokens and credential values should stay outside the KT artifact.

B.1 Fast Troubleshooting Map

Symptom	First place to inspect	Likely boundary
Login fails / browser says Failed to fetch	apps/web/src/lib/runtime-config.ts; backend/app/api/v1/auth.py	Frontend API base / proxy / CORS / auth
Redis unhealthy	backend/app/database/redis_client.py; docker-compose.yml	Redis availability or async client usage
Predictions slow	backend/app/services/ prediction_engine.py; market_data_service.py	Synchronous provider/network work in inference path
Terminal values stale	useTerminalData.ts; useSignalFeed.ts; backend/app/websocket/	REST refresh vs WebSocket stream
Backtest remains pending	api/v1/backtest.py; app/tasks/backtest_tasks.py; celery_app.py	Task registration / worker / result persistence
Model says not trained	backend/research/models/ + backend/app/models/model_artifact.py	Artifact activation/versioning / training state
Risk metrics 500	api/v1/portfolio.py; schemas/portfolio.py; models/portfolio.py	Response field/validation / portfolio state
Provider data missing	market_data_service.py; data_ingestion/*; providers/*	Provider health / fallback / rate limits
Hydration error in terminal	usePriceFeed.ts / useSignalFeed.ts / useSignalAlerts.ts	Server/client initial-state mismatch
Duplicate or missing terminal symbols	useTerminalData.ts; watchlist.py	Watchlist merge/deduplication

B.2 Safe Change Strategy

1. Capture the current Git diff and reproduce the issue before changing code.

2. Identify the owning boundary using the source-of-truth map in Appendix A.

3. Change the narrowest layer first; avoid embedding domain logic directly into route files when an existing service already owns it.

4. Run targeted backend/frontend checks, then the broader build/test workflow.

5. For market/ML changes, verify leakage, provider fallback and latency implications, not only functional output.

6. For trading/payments changes, explicitly test authorization, idempotency, limits, auditability and failure recovery.